Real-Time IMU Sensor Fusion on a Microcontroller

A Comparative Study Using Madgwick Orientation Filtering

Dimitrios Giannopoulos / up1100844
Georgios Paspalakis / up1101003

April 2026

1 Introduction

The goal of this project is to implement a real-time orientation estimator on a microcontroller using IMU measurements and to evaluate its performance through a comparative study. Specifically, the Madgwick gradient-descent orientation filter is implemented in three configurations: (i) a 6-axis IMU implementation using accelerometer and gyroscope data only, (ii) a 6-axis IMU implementation on a 9-axis module with disabled magnetometer data and (iii) a 9-axis MARG (Magnetic, Angular Rate, and Gravity) implementation that additionally incorporates the previously disabled magnetometer readings. Our sensor arrays consist of: (i) a low-cost Arduino onboard IMU and (ii) an industrial-grade MARG sensor array. The comparison provides insight into how sensor quality and the availability of a magnetic reference affect orientation estimation accuracy, drift behavior, and overall robustness.

2 Hardware Setup

The hardware setup consists of an Arduino Uno WiFi R2 and a Xsens MTi-3-5A-T.

2.1 Arduino Uno WiFi Rev2

The Arduino Uno WiFi Rev2 serves as the central processing unit. This board also features an onboard IMU, the LSM6DS3TR. It is a 6-axis inertial measurement unit comprising a 3-axis accelerometer and a 3-axis gyroscope. It does not include a magnetometer.

2.2 Xsens MTi-3-5A-T

The Xsens MTi-3-5A-T is an industrial-grade Attitude and Heading Reference System (AHRS) module from the MTi 1-series product line. It is a 9-axis MARG sensor array containing a 3-axis accelerometer, a 3-axis gyroscope, and a 3-axis magnetometer [2], along with an embedded low-power MCU that performs on-chip calibration (including temperature compensation, bias correction, and scale factor adjustment). The module is packaged in a JEDEC-PLCC-28 form factor.

2.3 Hardware Interfacing

The onboard IMU is interfaced directly with the microcontroller through the SPI bus. Its measurements are acquired at a configured output data rate of 104 Hz [4] through the `Arduino_LSM6DS3` library, without the need for any additional connections. The acquired data are then displayed on the Arduino Serial Monitor.

On the other hand, interfacing the Xsens module was not as straightforward. To interface it with the Arduino, a PLCC-28 to DIP-28 converter is used, thus avoiding direct soldering of the chip. The module clicks into the PLCC socket of the converter, and the bottom side of it provides two rows of 14 pins each, which can be plugged straight into a breadboard.

A very important aspect of the setup was the pin mapping of the converter. The datasheet did not specify which PLCC socket contacts corresponded to the DIP PINs, except for PIN1 which was indicated on the PCB, as shown in Figure 1. For this reason, a continuity test with a multimeter was carried out and the socket leads were mapped as shown in Figure 2.

In both the MTi module and the PLCC socket, PIN1 is the middle pin of the top row. However, the difference is the numbering direction. On the MTi module the numbering continues CCW whereas on the socket, the numbering continues CW. Figures 2a and 2b help visualize this difference.

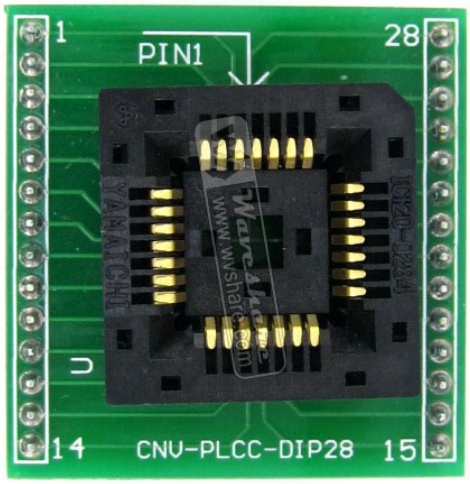
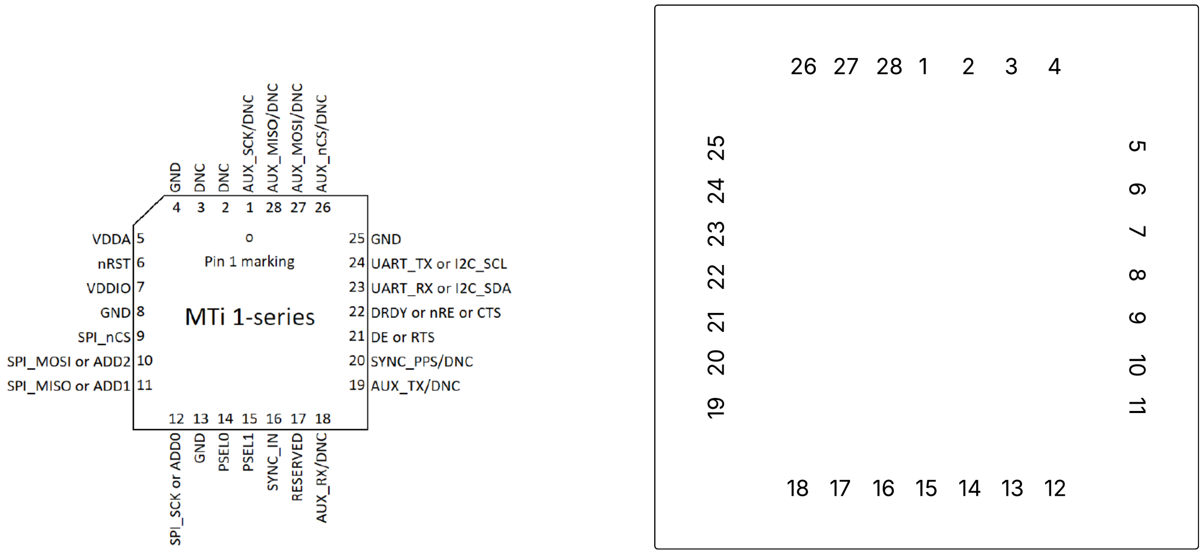


Figure 1: PIN1 indication on the PLCC-28 to DIP-28 converter PCB.



(a) MTi module pinout (CCW numbering).

(b) PLCC socket pinout (CW numbering).

Figure 2: Comparison of pin numbering between the MTi module and the PLCC socket.

The MTi 1-series modules support four peripheral interfacing modes: I^2C, SPI, UART half-duplex, and UART full-duplex [3]. The active interface is selected at startup by the logic state of two peripheral selection pins, PSEL0 (PIN14) and PSEL1 (PIN15). For this project, UART full-duplex was selected. This choice was motivated by several practical considerations: UART requires only two data lines (TX and RX), avoids the overhead of the MTSSP (MT Synchronous Serial Protocol) layer that is required for I^2C and SPI communication with the MTi 1-series, and allows standard Xbus protocol messages to be transmitted directly over the serial link without any additional framing. All the above made this mode ideal for the simple communication we wanted to establish with the MTi module in order to just gather the raw sensor data. Furthermore, the Arduino Uno WiFi R2 provides a dedicated hardware serial port (`Serial1` on digital pins 0 and 1 where the RX and TX lines are) [5].

Since the Arduino operates at 5V logic while the MTi-3 I/O voltage (VDDIO) is set to 3.3V, a resistive voltage divider (549 Ω / 989 Ω) was placed on the Arduino TX to MTi-3 UART RX line, stepping the signal down to approximately 3.22V. The reverse direction (MTi-3 TX to Arduino RX) requires no level shifting, as the Arduino's microcontroller recognizes 3.3V as a valid logic high. Additionally, the CTS (Clear To Send) pin was set to GND to permit continuous data transmission from the module without hardware flow control.

Table 1 summarizes the pin mapping used for full-duplex UART communication, including the pin remapping introduced by the PLCC-28 to DIP-28 converter.

Table 1: Pin mapping for full-duplex UART communication with the MTi-3 [2].

MTi PIN	**Socket PIN**	**Assignment**	**Note**
4	26	3.3V	On the corresponding Arduino pin
5	25	3.3V	
7	23	GND	
8	22	GND	
13	17	GND	
14	16	GND	Full-duplex UART
15	15	GND	
22	8	GND	No HW flow control
23	7	Arduino TX	Through voltage divider
24	6	Arduino RX	
25	5	GND	

3 Filtering Methodology

3.1 Overview of the Madgwick Filter

The Madgwick filter [1] is a computationally efficient orientation filter that fuses gyroscope, accelerometer and magnetometer data to produce a continuous quaternion estimate of orientation. It was selected for this project over Kalman-based approaches for three practical reasons: it requires only 1 or 2 tunable parameters with clear physical meaning, it is computationally efficient which is ideal in our case since we are performing the sensor fusion on the microcontroller, and it has been shown to perform better at reduced sampling rates.

The core idea of the filter is that orientation can be estimated in two fundamentally different ways: by integrating gyroscope angular rates or by observing the direction of known earth-frame reference fields. Fusing the two corrects the weaknesses of each. The gyroscope provides fast, smooth orientation updates but drifts over time. The accelerometer and magnetometer provide absolute orientation references but are noisy and sensitive to disturbances. The Madgwick filter combines these continuously, weighting the gyroscope during motion and allowing the reference measurements to gradually correct accumulated drift.

3.2 Orientation from Gyroscope

A gyroscope measures angular velocity ${}^{S}\boldsymbol{\omega} = [0,\ \omega_x,\ \omega_y,\ \omega_z]$ in the sensor frame. If the current orientation quaternion $\hat{q}$ is known, the rate of change of orientation can be expressed as a quaternion derivative:

$$ {}^{S}_{E}\dot{q}_{\omega,t} = \frac{1}{2}\, {}^{S}_{E}\hat{q}_{\mathrm{est},t-1} \otimes {}^{S}\boldsymbol{\omega}_t \tag{1} $$

The new orientation is obtained by numerically integrating this derivative over one time step Δt:

$$ {}^{S}_{E}q_{\omega,t} = {}^{S}_{E}\hat{q}_{\mathrm{est},t-1} + {}^{S}_{E}\dot{q}_{\omega,t}\,\Delta t \tag{2} $$

This approach is simple and fast, but has a fundamental problem: any constant offset in the gyroscope output (bias) gets integrated along with the true angular rate. Over time, this produces an orientation error that grows without bound. Also, the gyroscope alone needs to be given initial conditions to correctly estimate the orientation. It has no reference, it only knows how fast it is rotating, not where it is pointing.

3.3 Orientation from Accelerometer and Magnetometer

An alternative way to determine orientation is to observe how a sensor measures a known earth-frame reference field. If we know the true direction of a field in the earth frame, we can find the orientation $\hat{q}$ that best explains the sensor measurement. This is formulated as the following minimization problem:

$$\min_{{}^S_E\hat{q} \in \mathbf{R}^4} f\left({}^S_E\hat{q},\ {}^E\hat{d},\ {}^S\hat{s}\right) \tag{3}$$

where the objective function is defined as:

$$f\left({}^S_E\hat{q},\ {}^E\hat{d},\ {}^S\hat{s}\right) = {}^S_E\hat{q}^* \otimes {}^E\hat{d} \otimes {}^S_E\hat{q} - {}^S\hat{s} \tag{4}$$

Here ${}^S_E\hat{q}$ is the quaternion encoding the orientation of the earth frame relative to the sensor frame, which is basically what we are trying to find. ${}^E\hat{d}$ is the known direction of the reference field expressed in the earth frame. ${}^S\hat{s}$ is the normalized measurement of that same field as observed in the sensor frame. The term ${}^S_E\hat{q}^* \otimes {}^E\hat{d} \otimes {}^S_E\hat{q}$ is the predicted sensor-frame direction of the field given the current orientation estimate. The objective function f therefore measures the difference between the predicted measurement and the actual measurement. It equals zero only when the estimated orientation perfectly explains the observed field direction, thus meaning $\hat{q}$ is the true orientation of the earth frame relative to the sensor frame.

The minimization problem is solved using the gradient descent algorithm. Rather than iterating to convergence, Madgwick suggested that a single gradient descent step per time sample is sufficient, provided the step size does not exceed the physical rate of change of orientation. The gradient is computed analytically as:

$$\nabla f = J^T\left({}^S_E\hat{q},\ {}^E\hat{d}\right) f\left({}^S_E\hat{q},\ {}^E\hat{d},\ {}^S\hat{s}\right) \tag{5}$$

where J is the Jacobian of f with respect to ${}^S_E\hat{q}$. The normalized gradient $\nabla f / \|\nabla f\|$ points in the quaternion direction that most efficiently reduces the orientation error.

3.3.1 Orientation from the Accelerometer

For the gravitational field, the known earth-frame reference direction is ${}^E\hat{g} = [0,\ 0,\ 0,\ 1]$, which plays the role of ${}^E\hat{d}$ in the general formulation. The normalized accelerometer measurement ${}^S\hat{a} = [0,\ a_x,\ a_y,\ a_z]$ plays the role of ${}^S\hat{s}$. Substituting into equations (4) and (5) and simplifying analytically yields:

$$f_g(\hat{q},\ \hat{a}) = \begin{bmatrix} 2(q_2q_4 - q_1q_3) - a_x \\ 2(q_1q_2 + q_3q_4) - a_y \\ 2\left(\frac{1}{2} - q_2^2 - q_3^2\right) - a_z \end{bmatrix} \tag{6}$$

$$J_g(\hat{q}) = \begin{bmatrix} -2q_3 & 2q_4 & -2q_1 & 2q_2 \\ 2q_2 & 2q_1 & 2q_4 & 2q_3 \\ 0 & -4q_2 & -4q_3 & 0 \end{bmatrix} \tag{7}$$

However, minimizing f_g alone does not yield a unique solution. Intuitively, any rotation around the gravity axis (yaw) produces the exact same gravity measurement. The accelerometer cannot distinguish between orientations that differ only in heading. The solution surface has a minimum defined by a line rather than a point, leaving yaw completely unobservable.

3.3.2 Orientation from the Magnetometer

For the earth's magnetic field, the reference direction is ${}^E\hat{b} = [0,\ b_x,\ 0,\ b_z]$, which plays the role of ${}^E\hat{d}$. The magnetic field is modeled with only horizontal (b_x) and vertical (b_z) components to account for the inclination of the field. The normalized magnetometer measurement ${}^S\hat{m} = [0,\ m_x,\ m_y,\ m_z]$ plays the role of ${}^S\hat{s}$. The simplified analytical objective function and Jacobian are:

$$f_b\left(\hat{q},\ {}^E\hat{b},\ \hat{m}\right) = \begin{bmatrix} 2b_x\left(\frac{1}{2} - q_3^2 - q_4^2\right) + 2b_z(q_2q_4 - q_1q_3) - m_x \\ 2b_x(q_2q_3 - q_1q_4) + 2b_z(q_1q_2 + q_3q_4) - m_y \\ 2b_x(q_1q_3 + q_2q_4) + 2b_z\left(\frac{1}{2} - q_2^2 - q_3^2\right) - m_z \end{bmatrix} \tag{8}$$

$$J_b(\hat{q},\ {}^E\hat{b}) = \begin{bmatrix} -2b_zq_3 & 2b_zq_4 & -4b_xq_3 - 2b_zq_1 & -4b_xq_4 + 2b_zq_2 \\ -2b_xq_4 + 2b_zq_2 & 2b_xq_3 + 2b_zq_1 & 2b_xq_2 + 2b_zq_4 & -2b_xq_1 + 2b_zq_3 \\ 2b_xq_3 & 2b_xq_4 - 4b_zq_2 & 2b_xq_1 - 4b_zq_3 & 2b_xq_2 \end{bmatrix} \tag{9}$$

3.3.3 Orientation from Accelerometer and Magnetometer Combined

Using either field alone leaves rotational freedom around the axis parallel to that field. The solution is to stack both objective functions into a single objective function and solve the combined minimization problem. The only orientation that simultaneously satisfies both constraints is a unique point in quaternion space, provided $b_x \neq 0$ (This condition is always satisfied in practice, as the earth's magnetic field has a non-zero horizontal component everywhere on the surface of the earth. In Patras the magnetic inclination is approximately 55°):

$$f_{g,b}\left(\hat{q},\, \hat{a},\, {}^E\hat{b},\, \hat{m}\right) = \begin{bmatrix} f_g(\hat{q},\, \hat{a}) \\ f_b(\hat{q},\, {}^E\hat{b},\, \hat{m}) \end{bmatrix} \tag{10}$$

$$J_{g,b}\left(\hat{q},\, {}^E\hat{b}\right) = \begin{bmatrix} J_g^T(\hat{q}) \\ J_b^T(\hat{q},\, {}^E\hat{b}) \end{bmatrix} \tag{11}$$

The gradient $\nabla f_{g,b} = J_{g,b}^T\, f_{g,b}$ now drives the correction towards the unique orientation that simultaneously aligns both the gravitational and magnetic field measurements, giving a complete orientation estimate including heading.

3.4 IMU Implementation

In the IMU implementation (accelerometer + gyroscope, 6-axis), the two orientation estimates are fused at every time step. The gyroscope provides the orientation rate, and the accelerometer gradient correction is subtracted from it, scaled by the gain β:

$${}^S_E\dot{q}_{\text{est},t} = {}^S_E\dot{q}_{\omega,t} - \beta\, \frac{\nabla f_g}{\|\nabla f_g\|} \tag{12}$$

The orientation is then integrated and renormalized:

$${}^S_E q_{\text{est},t} = {}^S_E\hat{q}_{\text{est},t-1} + {}^S_E\dot{q}_{\text{est},t}\, \Delta t \tag{13}$$

The parameter β controls how much the filter trusts the accelerometer correction relative to the gyroscope. A larger β corrects orientation error faster but introduces more noise from the accelerometer. A smaller β gives a smoother output but is slower to correct drift. Madgwick suggests that β is derived from the estimated mean zero gyroscope measurement error $\tilde{\omega}_\beta$:

$$\beta = \sqrt{\frac{3}{4}}\,\tilde{\omega}_\beta \tag{14}$$

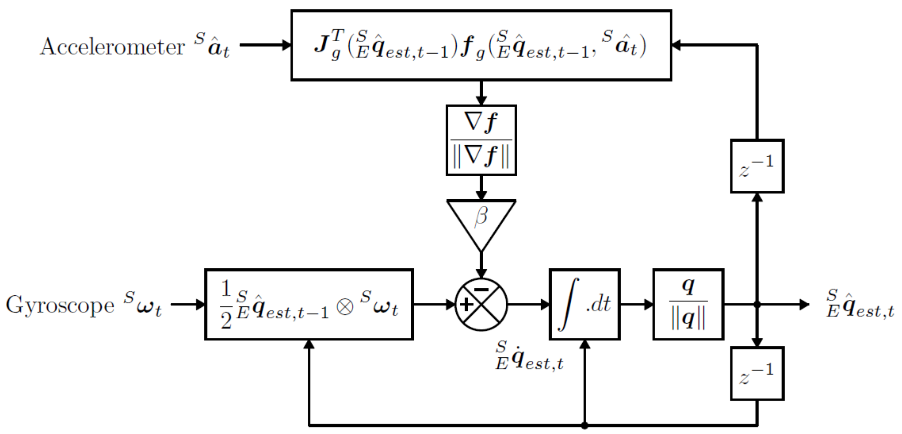


Figure 3: Block diagram of the Madgwick filter for the IMU implementation [1]. The gyroscope rate and the accelerometer gradient correction are fused to produce the quaternion estimate.

3.5 MARG Implementation

The MARG implementation (accelerometer + gyroscope + magnetometer, 9-axis) extends the IMU filter in two ways. Firstly, the magnetometer is added to the gradient correction to enable full orientation estimation, including yaw. In addition, a second parameter ζ is introduced to estimate and subtract the gyroscope zero-rate bias.

The fusion equation has the same structure as in the IMU case but uses the combined gradient from both accelerometer and magnetometer:

$$ {}^S_E\dot{q}_{\text{est},t} = {}^S_E\dot{q}_{\omega,t} - \beta \frac{\nabla f_{g,b}}{\|\nabla f_{g,b}\|} \tag{15}$$

The magnetic reference direction $\hat{b}$ is recomputed at every step by rotating the current magnetometer measurement into the earth frame and keeping only its horizontal and vertical components. This means local magnetic disturbances only corrupt the yaw estimate and do not affect roll or pitch:

$$ {}^E\hat{h}_t = {}^S_E\hat{q}_{\text{est},t-1} \otimes {}^S\hat{m}_t \otimes {}^S_E\hat{q}^*_{\text{est},t-1}, \qquad {}^E\hat{b}_t = \begin{bmatrix} 0 & \sqrt{h_x^2 + h_y^2} & 0 & h_z \end{bmatrix} \tag{16}$$

A simple gyroscope bias estimation loop is added. It works by converting the normalized gradient back into an angular error. Then, integrating that error with gain ζ, we get an estimate of the constant bias offset ${}^S\boldsymbol{\omega}_b$:

$$ {}^S\boldsymbol{\omega}_{b,t} = \zeta \sum_t 2\, {}^S_E\hat{q}^*_{\text{est},t-1} \otimes \dot{\hat{q}}_{\epsilon,t}\ \Delta t \tag{17}$$

This estimated bias is subtracted from the raw gyroscope reading before entering the integration step, so the corrected angular rate is used for orientation propagation. It is important to note that gyroscope bias estimation is only possible in the MARG implementation, because it requires a complete estimate ${}^S_E\hat{q}_{\text{est}}$ to convert the gradient error back into angular bias in each gyroscope axis.

Madgwick suggests that the second tunable parameter ζ is derived from the estimated gyroscope bias drift rate $\dot{\tilde{\omega}}_\zeta$:

$$ \zeta = \sqrt{\frac{3}{4}}\, \dot{\tilde{\omega}}_\zeta \tag{18}$$

Measuring the gyroscope bias drift rate is especially difficult, thus, selection of ζ is done by fine-tuning Madgwick's proposed value. In practice, ζ is set small so that the bias estimate converges slowly and does not react to transient motion. It is designed to remove the DC component of gyroscope error, not to track fast changes.

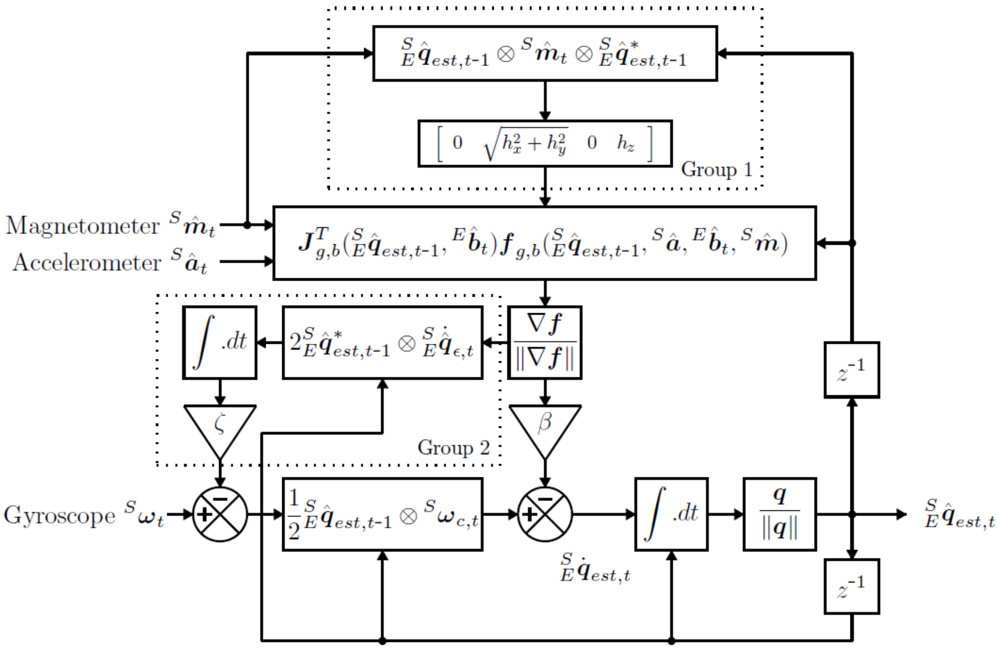


Figure 4: Block diagram of the Madgwick filter for the MARG implementation [1], including magnetic distortion compensation (Group 1) and gyroscope bias drift compensation (Group 2).

4 Results

4.1 Raw Sensor Data

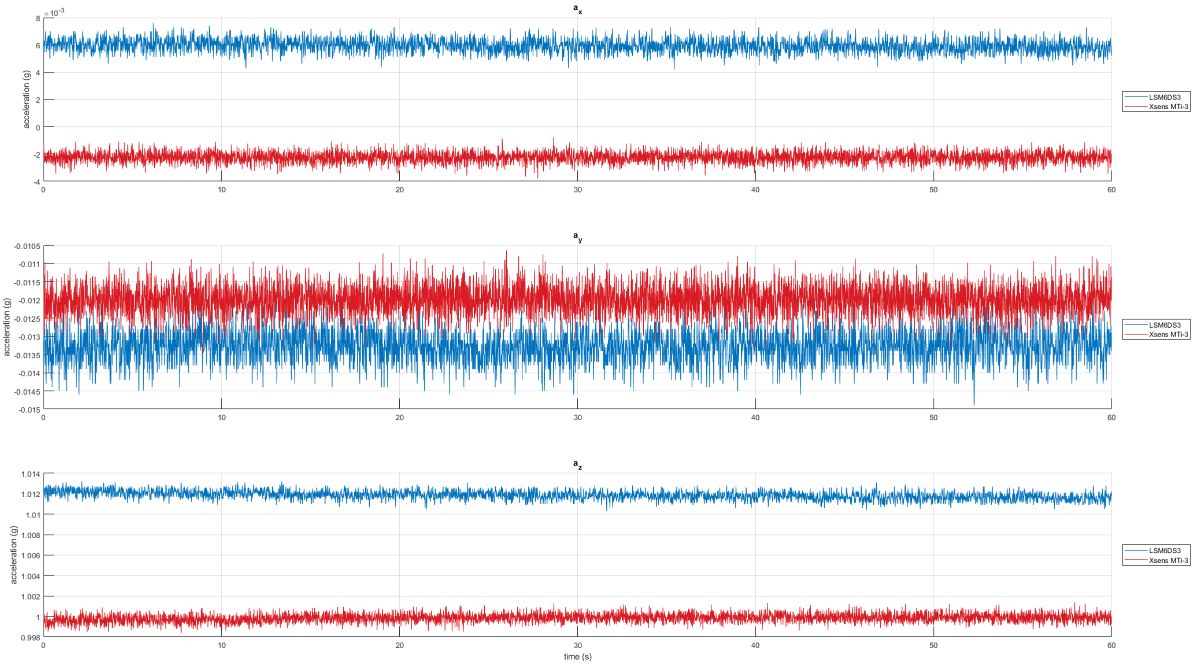


Figure 5: Raw accelerometer measurements from the LSM6DS3TR (blue) and the Xsens MTi-3 (red), with the sensor stationary on a flat surface for 60 s.

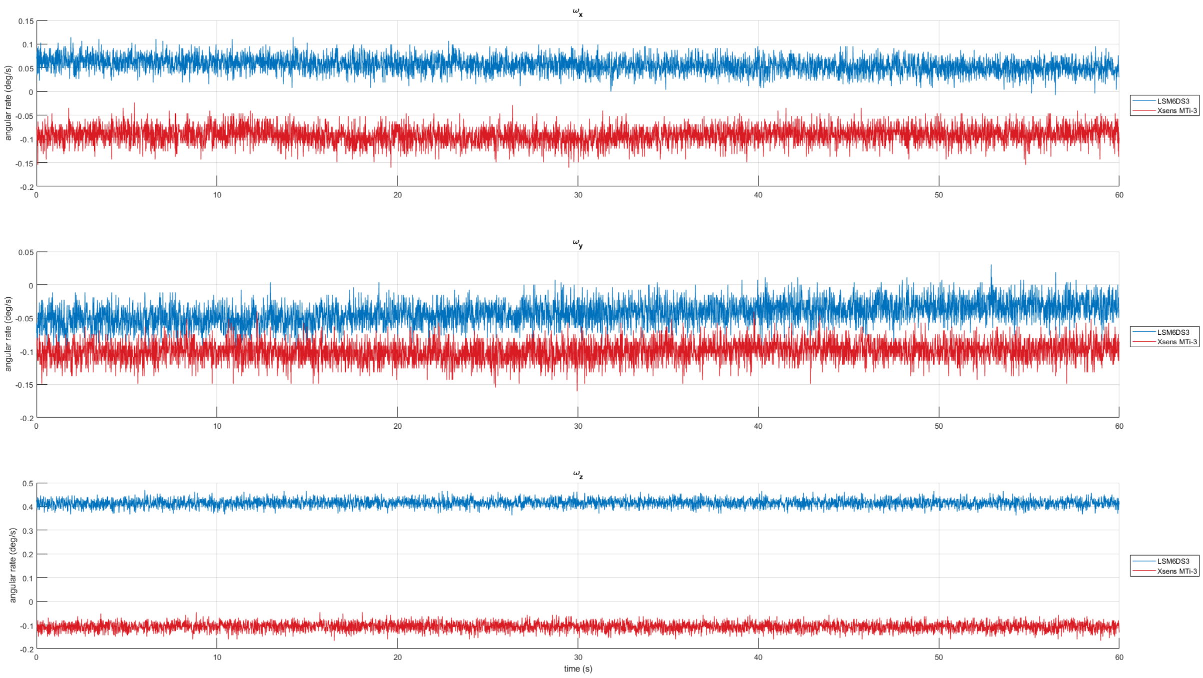


Figure 6: Raw gyroscope measurements from the LSM6DS3TR (blue) and the Xsens MTi-3 (red), with the sensor stationary on a flat surface for 60 s.

Table 2: Bias (mean) and noise (standard deviation) of the raw sensor measurements shown in Figures 5 and 6.

	LSM6DS3TR		Xsens MTi-3		
Axis	**Bias**	**Noise (σ)**	**Bias**	**Noise (σ)**	**Units**
a_x	0.00595	0.00047	-0.00225	0.00039	g
a_y	-0.01326	0.00043	-0.01199	0.00040	g
a_z	1.01185	0.00042	0.99985	0.00042	g
ω_x	0.05579	0.01637	-0.09189	0.01829	deg/s
ω_y	-0.04269	0.01740	-0.10036	0.01522	deg/s
ω_z	0.41478	0.01562	-0.10656	0.01766	deg/s
m_x	—	—	0.65870	0.00074	a.u.
m_y	—	—	-0.43497	0.00092	a.u.
m_z	—	—	-0.70021	0.00160	a.u.

4.2 Typical Filter Output

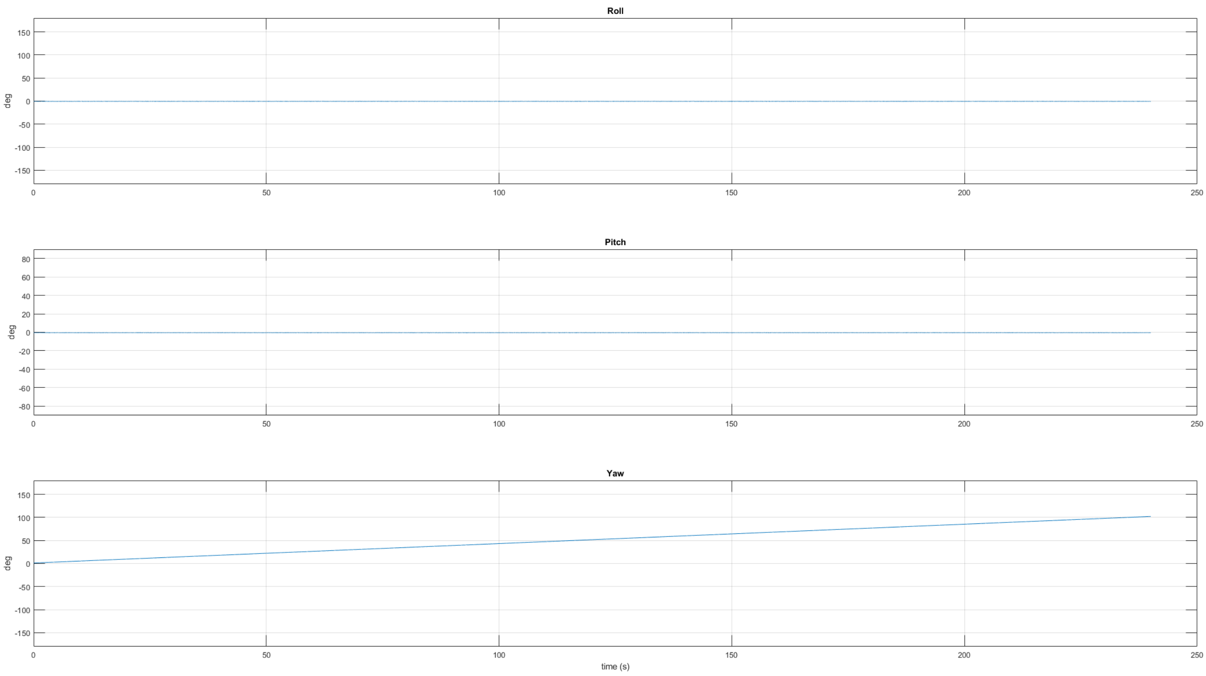


Figure 7: Roll, pitch, and yaw estimates from the Madgwick IMU filter running on the LSM6DS3TR.

Figure 8: Roll, pitch, and yaw estimates from the Madgwick IMU filter running on the Xsens MTi-3.

Figure 9: Roll, pitch, and yaw estimates from the Madgwick MARG filter running on the Xsens MTi-3.

Table 3: Mean and standard deviation of roll, pitch, and yaw from the filter outputs shown in Figures 7–9. Statistics computed over the post-convergence window ($t > 10$ s).

Configuration	Roll		Pitch		Yaw		Units
	Mean	σ	Mean	σ	Mean	σ	
LSM6DS3TR IMU	-0.6843	0.0407	-0.4542	0.0424	—	—	deg
Xsens MTi-3 IMU	-0.2663	0.0397	-0.1124	0.0382	—	—	deg
Xsens MTi-3 MARG	-0.4076	0.0271	-0.0939	0.0326	28.7127	0.0272	deg

Table 4: Yaw drift rate for the three filter configurations, measured as the slope of a linear fit to the yaw output after the initial 10 s convergence window.

Configuration	Drift rate (deg/s)	Total drift over 240 s (deg)
LSM6DS3TR IMU	0.4203	100.85
Xsens MTi-3 IMU	-0.1183	-28.38
Xsens MTi-3 MARG	≈ 0	≈ 0

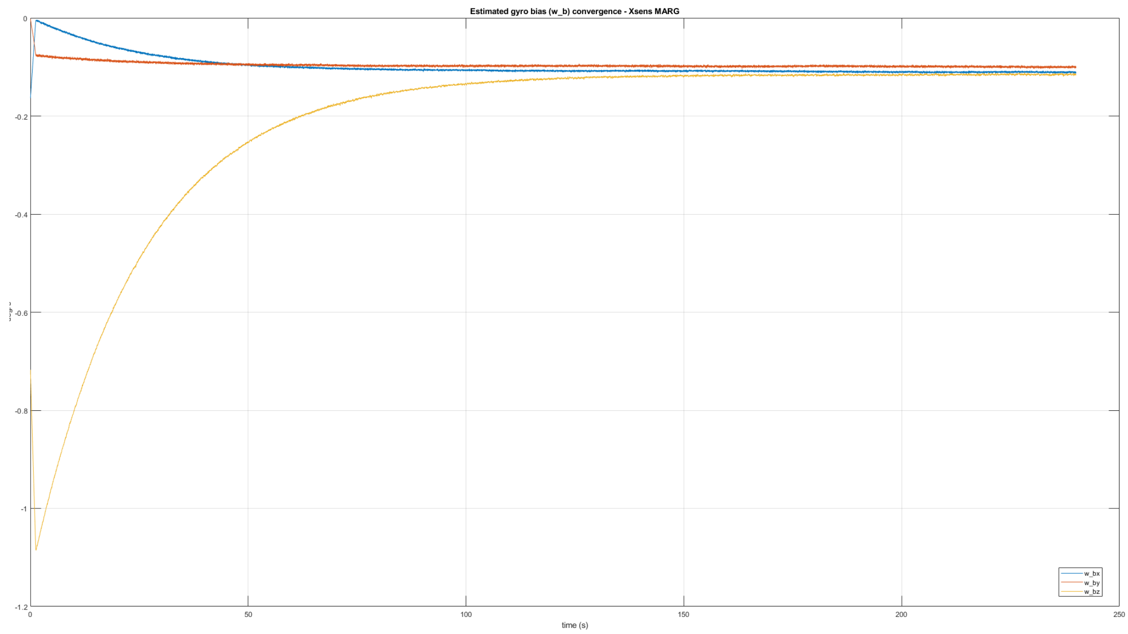


Figure 10: Convergence of the estimated gyroscope bias components w_{bx}, w_{by}, w_{bz} in the Madgwick MARG filter.

Table 5: Settled estimated gyroscope bias from the MARG filter (mean over the last 30 s of the 4-minute run shown in Figure 10), compared against the raw gyroscope bias measured independently in Table 2.

Axis	Filter-estimated bias (deg/s)	Raw-measured bias (deg/s)	Difference (deg/s)
w_{bx}	-0.11041	-0.09189	-0.01852
w_{by}	-0.09974	-0.10036	0.00062
w_{bz}	-0.11547	-0.10656	-0.00891

4.3 Effect of β on MARG Filter Output

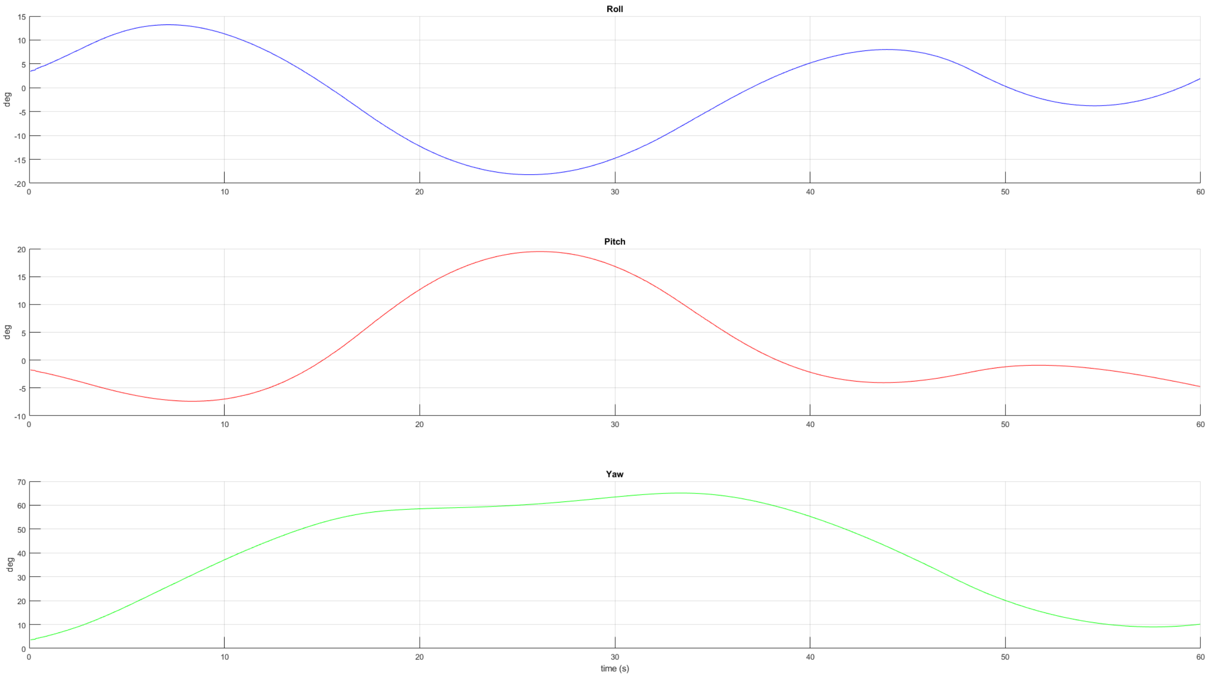


Figure 11: Roll, pitch, and yaw estimates from the MARG filter with $\beta = 0.00044$

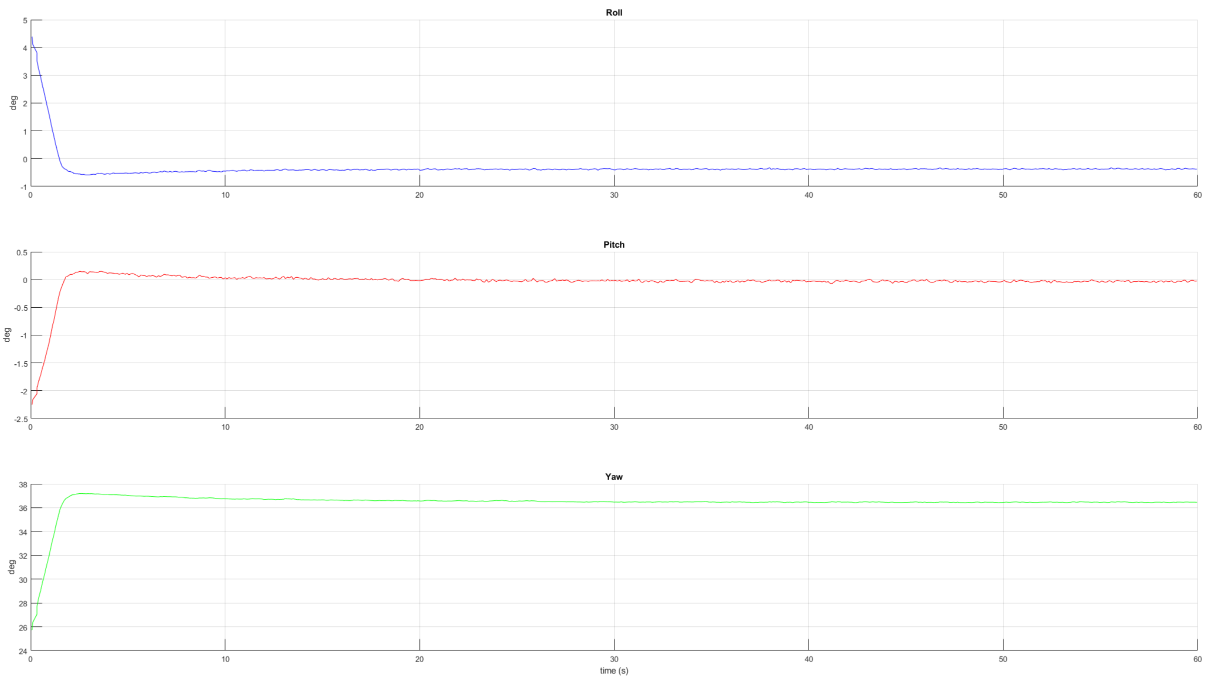


Figure 12: Roll, pitch, and yaw estimates from the MARG filter with $\beta = 0.05$

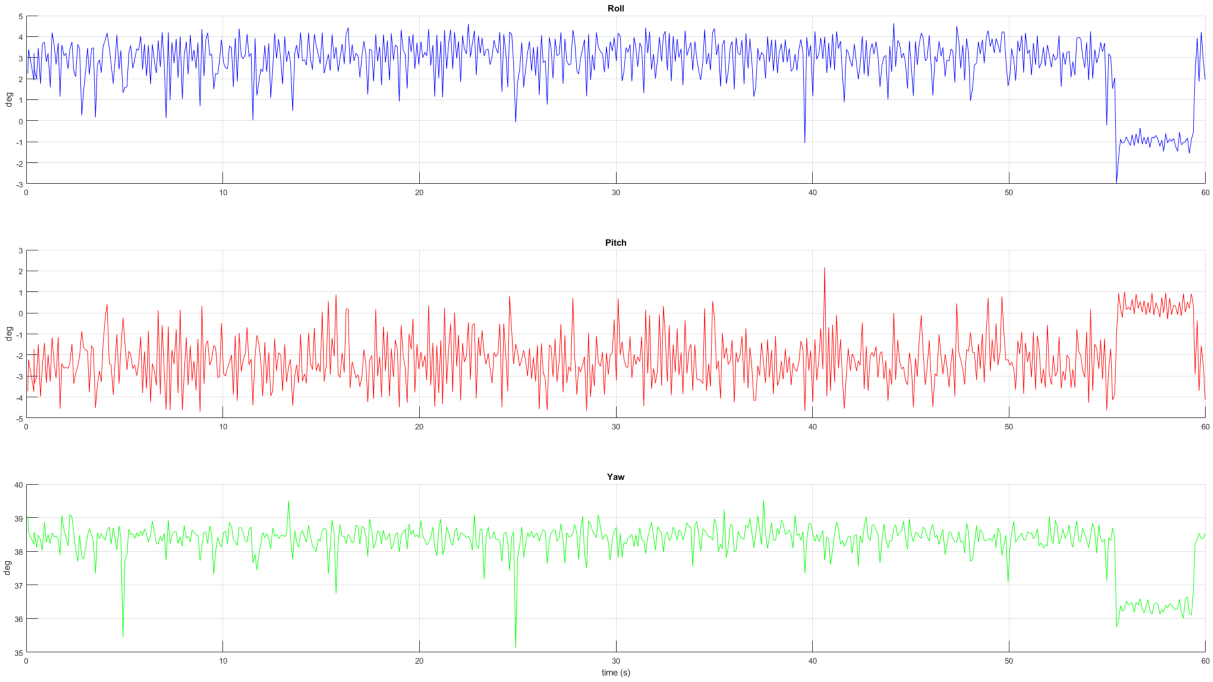


Figure 13: Roll, pitch, and yaw estimates from the MARG filter with $\beta = 5$

Table 6: Effect of β on MARG filter steady-state RPY noise (σ, computed over the last 30 s) and convergence time (last time the signal exits the mean $\pm 5\sigma$ band, computed per axis from the last 30 s). $\zeta = 0.005$ is held fixed across all runs.

	Roll		**Pitch**		**Yaw**	
$\boldsymbol{\beta}$	$\boldsymbol{t_s}$**(s)**	$\boldsymbol{\sigma}$**(deg)**	$\boldsymbol{t_s}$**(s)**	$\boldsymbol{\sigma}$**(deg)**	$\boldsymbol{t_s}$**(s)**	$\boldsymbol{\sigma}$**(deg)**
0.02	23.71	0.0081	25.50	0.0078	25.11	0.0116
0.03	19.09	0.0099	16.68	0.0095	20.38	0.0149
0.04	15.57	0.0124	15.81	0.0134	27.22	0.0185
0.05	11.79	0.0151	13.77	0.0144	26.01	0.0207
0.06	12.01	0.0162	9.19	0.0171	20.72	0.0193
0.07	5.40	0.0190	10.12	0.0173	—	0.0345
0.08	1.08	0.0218	0.83	0.0214	13.23	0.0256

4.4 Effect of ζ on MARG Filter Gyroscope Bias Estimation

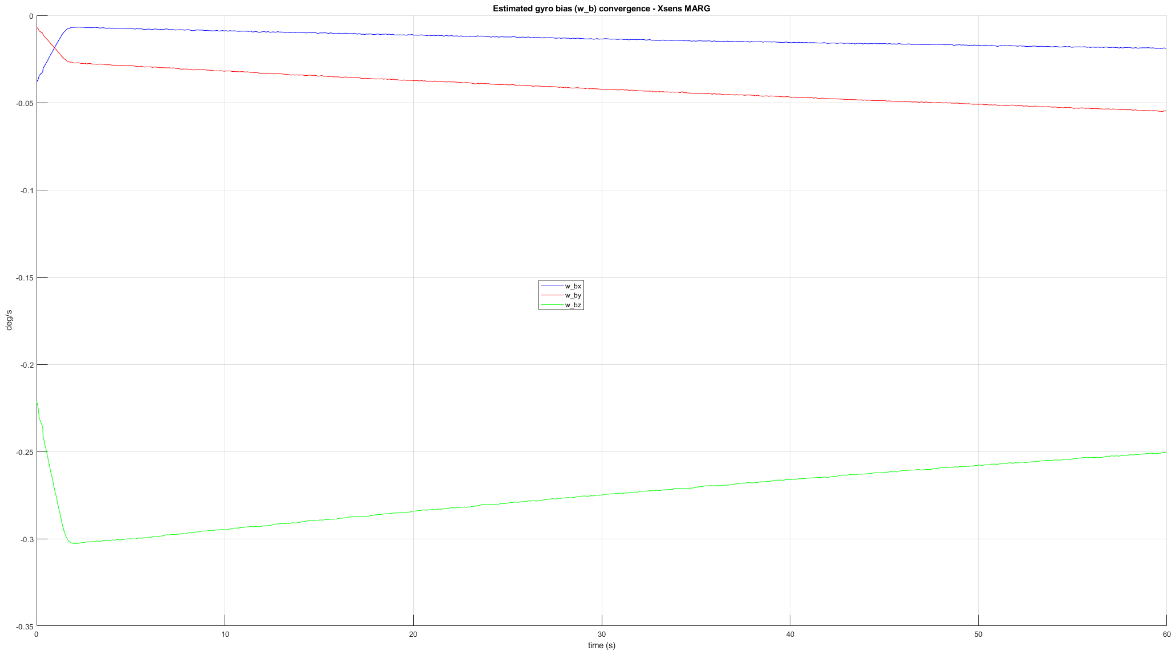


Figure 14: Estimated gyroscope bias ω_b with $\beta = 0.06$, $\zeta = 0.001$.

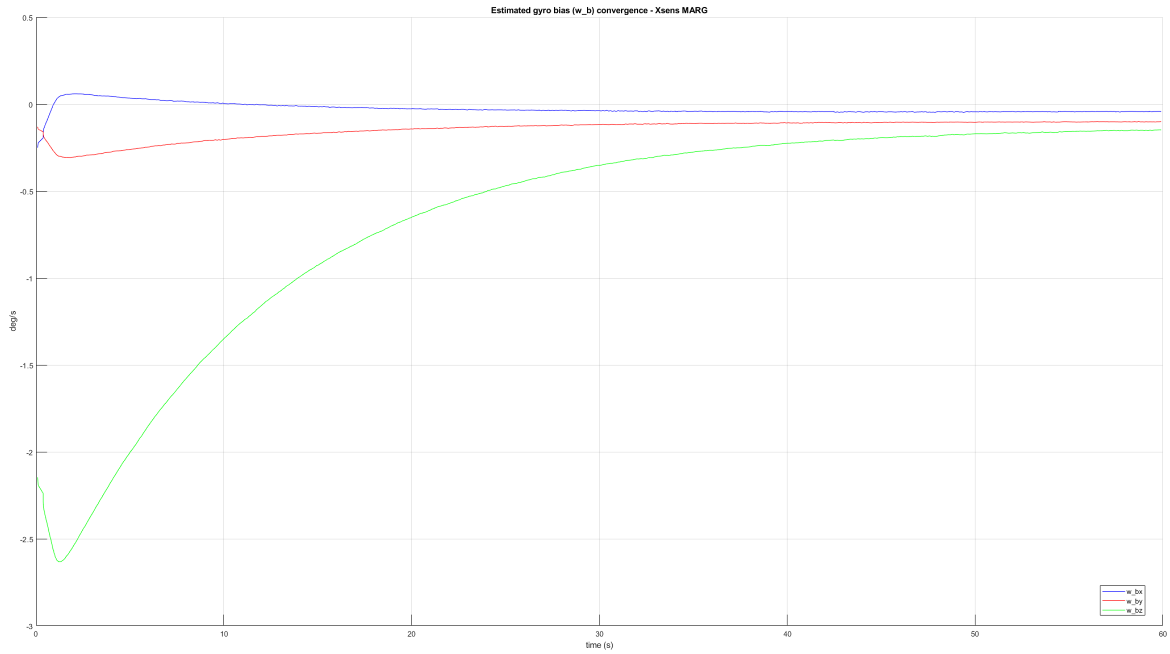


Figure 15: Estimated gyroscope bias ω_b with $\beta = 0.06$, $\zeta = 0.005$.

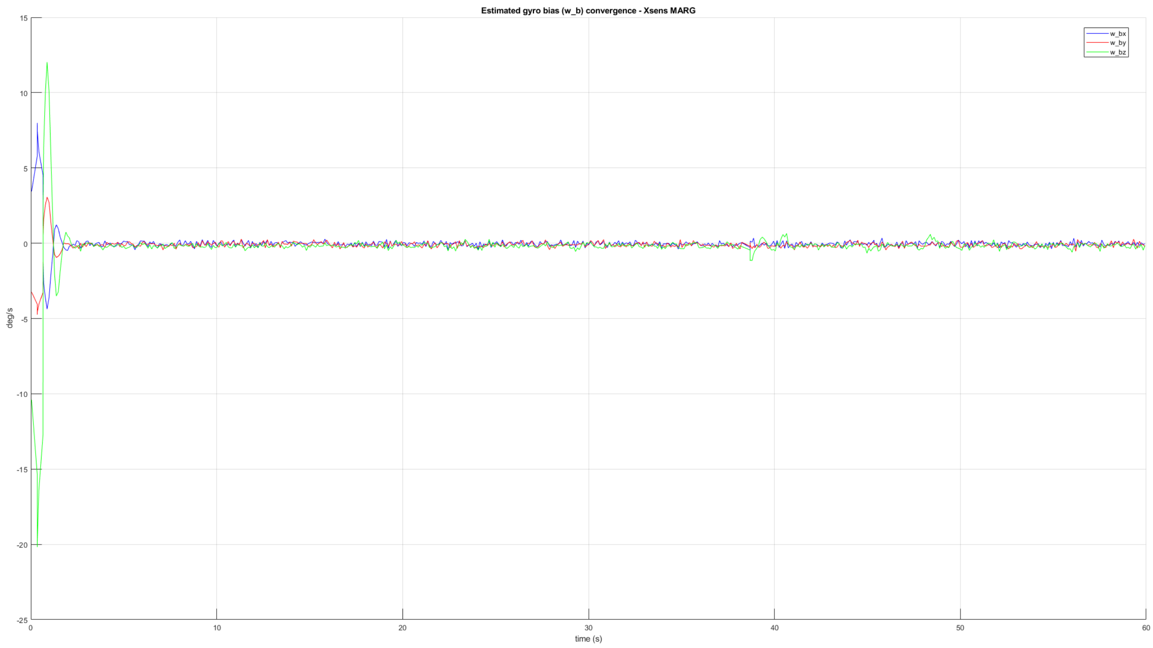


Figure 16: Estimated gyroscope bias ω_b with $\beta = 0.06$, $\zeta = 0.05$.

4.5 Effect of Update Rate on MARG Filter Output

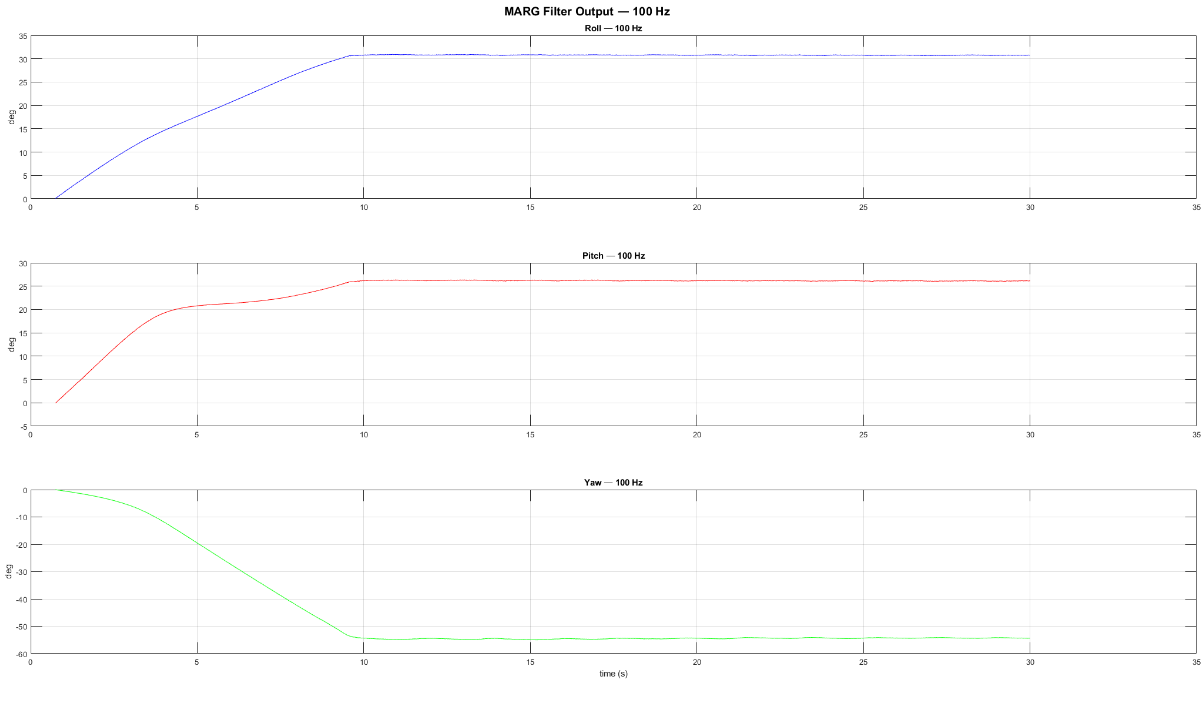


Figure 17: MARG filter RPY output at 100 Hz.

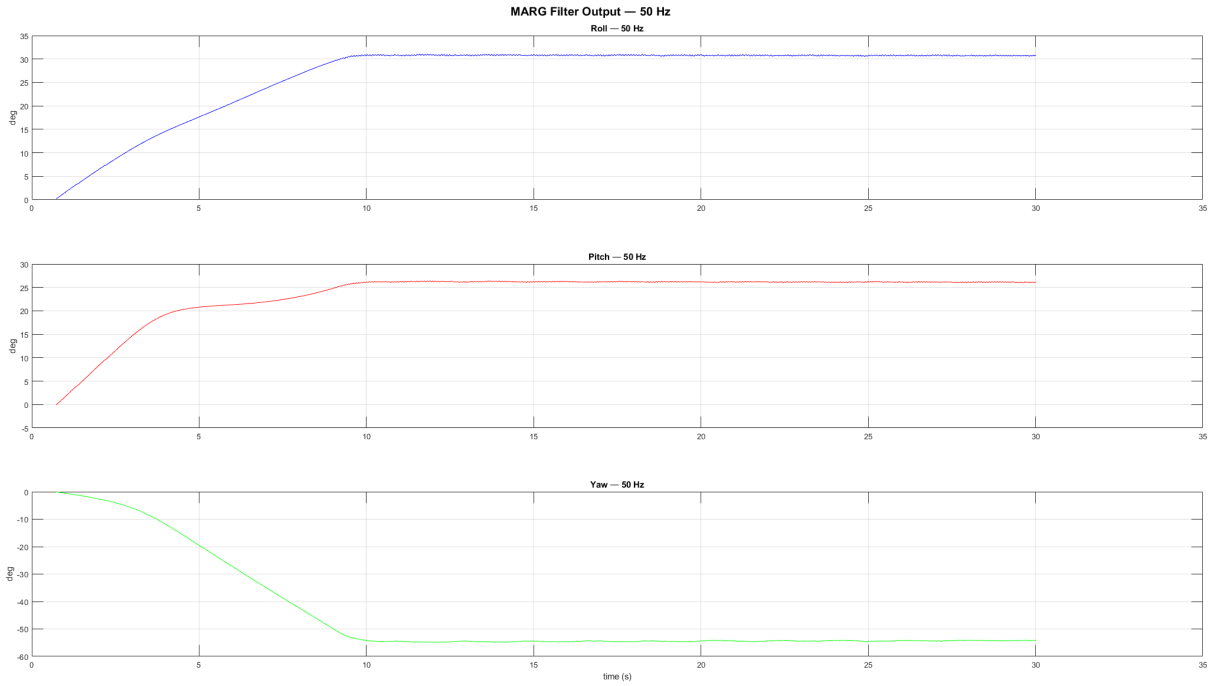


Figure 18: MARG filter RPY output at 50 Hz.

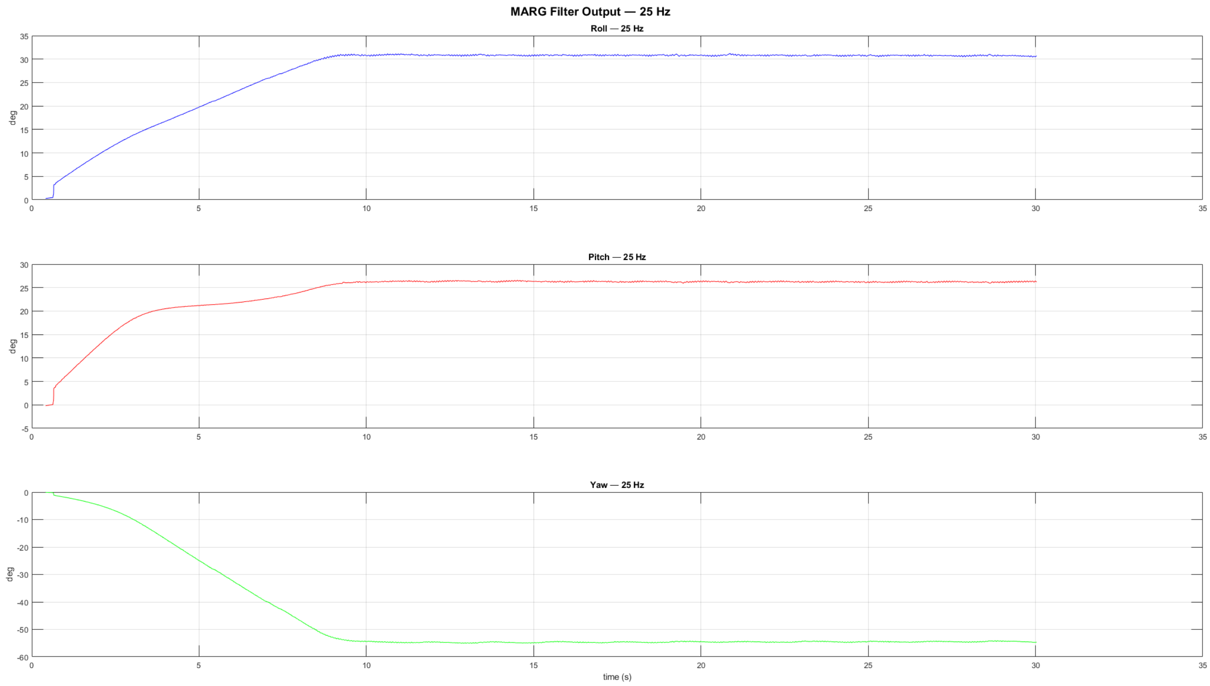


Figure 19: MARG filter RPY output at 25 Hz.

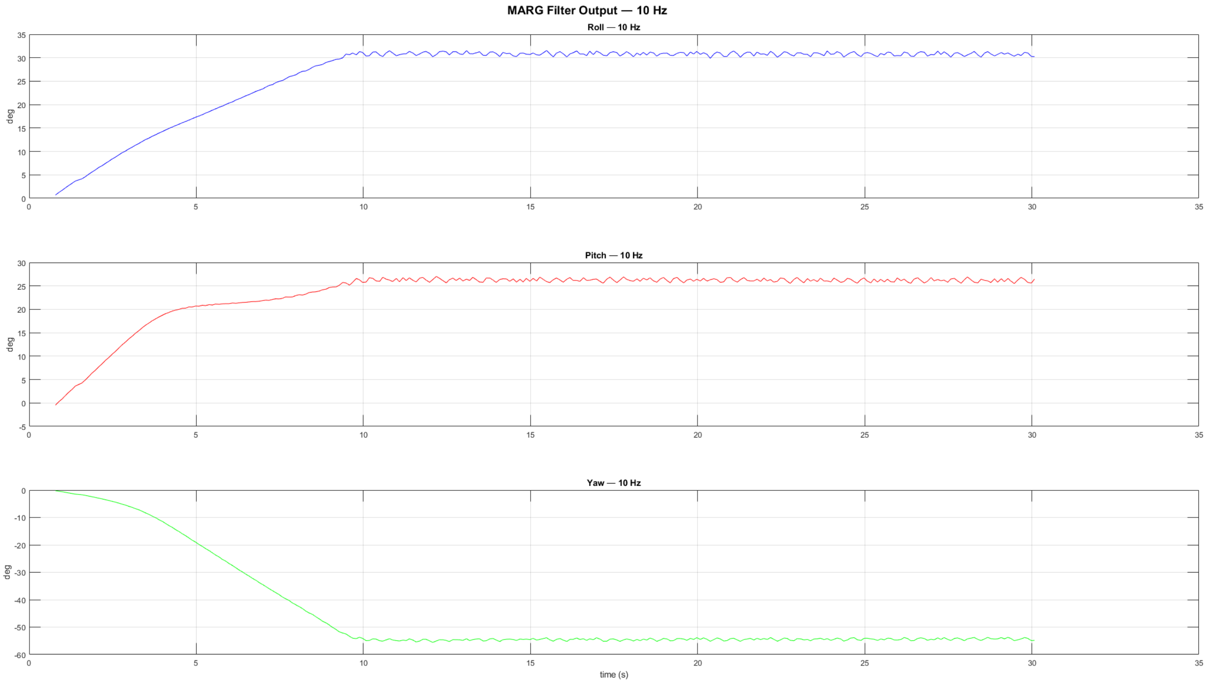


Figure 20: MARG filter RPY output at 10 Hz.

Table 7: Steady-state RPY noise (σ, computed over the last 15 s) as a function of filter update rate. $\beta = 0.06$, $\zeta = 0.005$ throughout.

Update rate (Hz)	$\boldsymbol{\sigma_{\text{roll}}}$ **(deg)**	$\boldsymbol{\sigma_{\text{pitch}}}$ **(deg)**	$\boldsymbol{\sigma_{\text{yaw}}}$ **(deg)**
100	0.0490	0.0624	0.1857
50	0.0953	0.0835	0.1639
25	0.1374	0.1262	0.1746
10	0.3712	0.3660	0.3859

5 Brief Analysis

5.1 Sensor Noise

The raw sensor data recorded in Section 4.1 shows that the two platforms have very similar noise characteristics.

For the gyroscope, the MTi-3 has a noise standard deviation of approximately 0.017 °/s per axis, while the LSM6DS3TR shows approximately 0.016 °/s. The bias magnitudes on x axis and y axis are also very similar with the LSM6DS3TR appearing slightly better. What is really important and makes the MTi-3 stand out is the z-axis bias: the LSM6DS3TR z-axis bias is 0.41 °/s, roughly four times larger than the MTi-3's 0.11 °/s.

For the accelerometer, both sensors show similar noise floors around 0.0004 g per axis, but the MTi-3's z-axis reading of 0.99985 g is closer to the expected 1 g than the LSM6DS3TR's 1.01185 g, indicating better sensor resolution.

It should be noted that prior to the raw data analysis, the gyroscope full-scale range of the onboard LSM6DS3TR had to be reconfigured from the library default of ±2000 °/s to ±125 °/s, while keeping the sampling frequency unchanged. In the default range and for the sensor stationary, the raw readings collapsed to only three distinct quantization levels, making any meaningful bias or noise characterization impossible. Reducing the full-scale range increased the resolution of the ADC output, revealing the true noise distribution of the sensor at rest.

5.2 Yaw Drift

Yaw drift is the fundamental limitation of the 6-axis IMU implementation as we explained in Section 3.

Any non-zero mean gyroscope bias on the z-axis integrates linearly into growing yaw error. From the raw data, the LSM6DS3TR has a z-axis gyroscope bias of $0.41\,°/\text{s}$, which would produce a yaw drift of approximately $24.6\,°/\text{min}$ if uncorrected, consistent with what is observed in the IMU filter output in Section 4.2. It is worth noting that even the MTi-3 operating in 6-axis IMU mode (without the magnetometer) would exhibit yaw drift, though significantly slower: its z-axis bias of $0.11\,°/\text{s}$ corresponds to approximately $6.6\,°/\text{min}$.

The calculated yaw drifts are consistent with the raw sensor analysis in Section 4.1. We showed that the MTi-3 z-axis gyroscope bias is roughly four times smaller than that of the LSM6DS3TR, which directly corresponds to the reality on our 4 minute stationary test.

The MARG implementation resolves this entirely by adding the magnetometer, which provides a second reference direction not parallel to gravity eliminating yaw drift. This is directly confirmed by the MARG steady-state yaw standard deviation of $\sigma_\psi \approx 0.02\,°$ in Section 4.2, compared to unbounded drift in the IMU case.

Yaw is non-zero in the MARG implementation because the sensor was placed flat on the table in an arbitrary orientation, with no attempt to align it with magnetic north. The non-zero yaw value simply reflects the actual heading of the sensor relative to the earth's magnetic field reference frame, which is expected and correct behaviour.

An important note: for this typical filter outputs test, the IMU and MARG filter implementations were used with Madgwick's proposed gains, which are $\beta = 0.0766$ and $\zeta = 0.00307$.

5.3 Tuning β

The parameters β and ζ are not fully independent as discussed in Section 3.5. ζ uses the same gradient signal as β, so changing β affects the signal driving the bias estimate. The approach taken here was to first identify a reasonable starting point near Madgwick's suggested values. We chose $\beta = 0.05$ and $\zeta = 0.005$, and after verifying that the filter output looked reasonable, we then performed a sweep on β with ζ held fixed at 0.005.

As a lower bound reference, β was first computed using Madgwick's formula (Equation 14) with $\tilde{\omega}_\beta$ set to the RMS value of the MTi-3 gyroscope biases from Section 4.1, giving $\tilde{\omega}_\beta \approx 0.029\,°/\text{s}$ and a theoretical $\beta \approx 0.00044$. The corresponding filter output is shown in Figure 11: as expected, the orientation estimate barely converges, exhibiting slow oscillation with no meaningful settling.

A decade sweep was then performed across $\beta \in \{0.005, 0.05, 0.5, 5\}$ to identify the useful order of magnitude. The $\beta = 0.05$ case (Figure 12) was the only one that showed clean convergence and low steady-state noise. The $\beta = 5$ case (Figure 13) confirmed that excessively high values introduce unacceptable noise into all three angles. This established $\beta \sim 0.05$ as the correct order of magnitude to work within.

A finer sweep was then conducted across $\beta \in \{0.02, 0.03, 0.04, 0.05, 0.06, 0.07, 0.08\}$ with results summarised in Table 6. The data confirm the expected tradeoff: settling time decreases with increasing β, while steady-state standard deviation increases. For the purposes of this project the value $\beta = 0.06$ was selected as the best tradeoff, offering fast roll and pitch convergence (within approximately 10 s) and a low steady-state standard deviation of $0.016\,°$.

5.4 Tuning ζ

With $\beta = 0.06$ fixed as the selected operating point, the focus shifts to ζ and its effect on the gyroscope bias estimate convergence. The parameter ζ has negligible effect on steady-state RPY noise, its sole purpose is to drive $\hat{\omega}_b$ toward the true gyroscope bias over time.

Just like before, a decade sweep was performed starting from $\zeta = 0.0005$ and moving up to $\zeta = 0.5$. From this sweep, three representative cases are shown in Figures 14–16.

At $\zeta = 0.0005$ (Figure 14), the bias estimate is extremely slow — none of the three axes come close to converging within the 60-second window. This is not a usable operating point for any application where bias compensation is expected to take effect within a reasonable time.

At $\zeta = 0.005$ (Figure 15), all three axes converge cleanly with low noise. It is worth noting that this convergence is approximately three times faster than what was observed in the typical filter output of Section 4.2, where Madgwick's original $\beta = 0.0766$ and $\zeta = 0.00307$ were used — which is expected, as the higher ζ here intentionally trades convergence speed for a practically unoticable noisier steady-state estimate.

At $\zeta = 0.5$ (Figure 16), convergence is very fast. All axes settle in under 5 seconds. However, the bias estimates are noticeably noisier. This level of noise in $\hat{\omega}_b$ would propagate into the corrected gyroscope signal and degrade orientation quality possible causing drift amongst all axes.

The value $\zeta = 0.005$ was therefore selected. The bias values after 60 seconds are: $\hat{\omega}_{bx} \approx -0.05\,^\circ/\mathrm{s}$, $\hat{\omega}_{by} \approx -0.10\,^\circ/\mathrm{s}$, and $\hat{\omega}_{bz} \approx -0.15\,^\circ/\mathrm{s}$ (mean over last 5 seconds). They are in close agreement with the raw MTi-3 gyroscope bias measurements from Section 4.1, confirming that the estimator is converging towards the correct values.

5.5 Update-Rate Considerations

To ensure consistent conditions across all update-rate tests, the MTi-3 was placed in a fixed tilted position, rotated about all three axes simultaneously, and held stationary in that orientation for the entire set of experiments. This ensured that the filter had a non-trivial initial orientation error to converge from in all cases, making the convergence behaviour comparable across runs.

The update rate was adjusted by modifying two parameters in the Arduino code: the `deltat` constant, which sets the integration time step in the filter equations, and the `outCfg` register configuration of the MTi-3, which controls the output data rate of the sensor.

An attempt was made to push the update rate to 200 Hz. This was unsuccessful as the MATLAB logging script found that even after setting the update rate to 200 Hz, the measured frequency of the incoming data was still ∼100 Hz. The likely cause was that the onboard magnetometer of the MTi-3 is not capable of outputting at this rate. The experiments were therefore limited to a maximum of 100 Hz.

The results are summarized in Table 7 and the corresponding RPY plots are shown in Figures 17–20. At 50 Hz, the filter output is virtually indistinguishable from 100 Hz, the steady-state standard deviation increases only slightly and convergence behaviour is identical.

At 25 Hz, a noticeable increase in steady-state noise becomes visible in the plots, with roll and pitch standard deviation roughly doubling compared to 100 Hz. The filter still converges cleanly, but the output is visibly rougher. At 10 Hz, the noise level is significantly higher, approximately an order of magnitude worse than 100 Hz, and while convergence is still achieved, which is in itself a demonstration of the filter's robustness at low sampling rates, the output quality would be unsuitable for any motion-sensitive application.

Notably, the convergence time is broadly consistent across all four update rates. What seems to change with update rate is purely the steady-state noise floor, not the speed of initial lock-on.

6 Real-Time Orientation Viewer

A real-time 3D orientation viewer was implemented in MATLAB to visually demonstrate the filter output. The viewer renders an animated 3D box whose orientation tracks the physical sensor in real time, updated at 100 Hz via the serial link.

The visualisation is built on two helper classes, `HelperOrientationViewer` and `HelperBox`, obtained from MathWorks [6]. These classes handle the 3D rendering and quaternion-to-rotation-matrix conversion internally. The live MATLAB script reads the quaternion streamed from the Arduino, constructs a `quaternion` object, and passes it directly to the viewer each loop iteration.

The MTi-3 sensor frame uses a right-handed coordinate system with Z pointing up, X pointing forward, and Y pointing left, as shown in Figure 21. The `HelperBox` viewer, however, operates in the NED (North-East-Down) convention, where Z points down. When the sensor was aligned with magnetic north and the viewer was first tested without any correction, a clear 180° mismatch was observed. Rotating the physical sensor counter-clockwise produced a clockwise rotation on screen, due to the opposite Z axis directions.

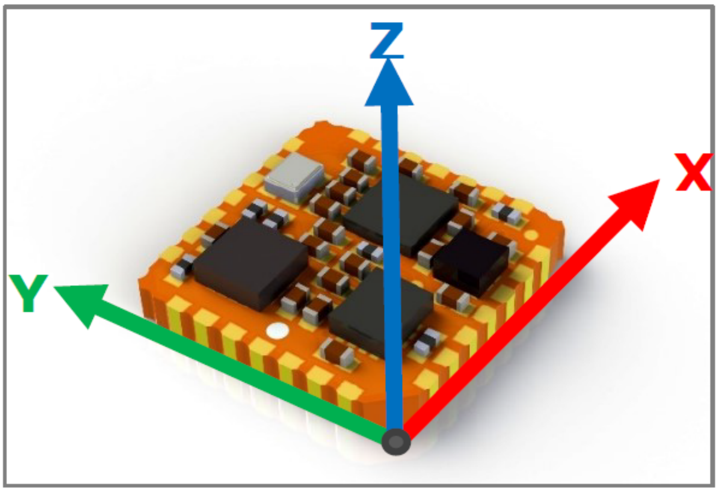


Figure 21: Default sensor fixed coordinate system of the MTi-3. This differs from the NED convention used by the orientation viewer, requiring a frame correction.

To correct this, a fixed frame correction quaternion representing a 180° rotation about the X axis was applied to every incoming quaternion before passing it to the viewer:

$$q_{\text{display}} = q_{\text{align}} \otimes q, \qquad q_{\text{align}} = [0,\ 1,\ 0,\ 0] \tag{19}$$

This correction is constant and does not affect the orientation estimate itself. After applying the correction, all three axes responded correctly to physical rotation of the sensor.

It should be noted that the roll, pitch, and yaw displayed by the viewer are defined relative to the sensor's own body frame axes (X, Y, Z as shown in Figure 21), not relative to any fixed absolute reference. Roll corresponds to rotation about the sensor X axis, pitch about Y, and yaw about Z.

A live demonstration was recorded and can be viewed by clicking here.

7 Conclusions

The magnetometer is essential for complete orientation estimation. The 6-axis IMU implementation cannot observe yaw, as gravity provides no reference for rotation about the vertical axis. Yaw is therefore determined entirely by gyroscope integration and drifts without bound. The 9-axis MARG implementation resolves this by incorporating the earth's magnetic field as a second reference, providing a stable and drift-free heading estimate under all tested conditions. For any application requiring absolute orientation, the magnetometer is not optional.

The Madgwick filter is well suited for embedded real-time orientation estimation. With only a couple hundred arithmetic operations per update for MARG implementation, the filter runs comfortably on an 8-bit microcontroller. The two tunable parameters have clear physical interpretations and can be tuned systematically, without the complexity of Kalman-based covariance tuning.

Sensor quality does not scale linearly with cost. The MTi-3 is roughly one hundred times more expensive than the LSM6DS3TR, yet in terms of the metrics that matter for this specific application — the resulting orientation accuracy — the difference is measurable but far from proportional to the price gap. For applications where moderate orientation accuracy is sufficient, a carefully configured low-cost IMU with a well-tuned Madgwick filter can deliver results that are close to those of a significantly more expensive industrial module.

References

[1] S. O. H. Madgwick, "An efficient orientation filter for inertial and inertial/magnetic sensor arrays," Internal Report, April 2010.

[2] Xsens Technologies B.V., "MTi 1-series Datasheet," Document MT0512P.

[3] Xsens Technologies B.V., “MT Low Level Communication Protocol Documentation,” Document MT0101P, Revision 2020.A.

[4] ST Microelectronics, “LSM6DS3 Datasheet,” 2017.

[5] Arduino, “Arduino Uno WiFi Rev2 Documentation,” `https://docs.arduino.cc/hardware/uno-wifi-rev2`.

[6] MathWorks, *HelperOrientationViewer and HelperBox*, MATLAB File Exchange, 2018.